I italicised the sql part

Part I: Short Answer

Q1)

= NULL never evaluates to TRUE; NULL comparisons using = always return UNKNOWN, not TRUE or FALSE, so no row passes the filter. The correct query uses IS NULL:

SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;

NULL represents the absence of a value, so it cannot be equal to, greater than, or less than anything, including itself. Aggregates like COUNT(col), AVG, SUM silently skip NULLs for the same reason: including an unknown value would make the result meaningless.

Q2)

The student is correct, DISTINCT is redundant here. GROUP BY department already guarantees one row per department, so there are no duplicate (department, COUNT(*)) pairs to eliminate. DISTINCT would be applied after grouping to deduplicate the result set, but since each group produces exactly one row, it has no effect.

Q3)

The difference means some customers have no matching rows in the Order table. LEFT JOIN returns all rows from Customer regardless of a match; INNER JOIN drops customers with no orders. The extra rows in the LEFT JOIN result are customers whose Order columns are NULL, i.e., customers who have never placed an order.

Q4)

The query is invalid because employee_name is neither in the GROUP BY clause nor wrapped in an aggregate function. After grouping by department, each group contains multiple employees, so there is no single well-defined value to display for employee_name. SQL would have to arbitrarily pick one row's name, which is non-deterministic and meaningless. The fix:

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department, employee_name;
```

Q5)

WHERE is evaluated before GROUP BY, so aggregate functions like AVG(salary) don't exist yet when WHERE runs, the groups haven't been formed. HAVING is evaluated after grouping and is the correct clause for filtering on aggregates:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6)
(a) Products priced above the overall average are a non-correlated subquery.

The overall average is a single value computed once and reused; it doesn't depend on the current row.

```
SELECT * FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Employees earning above their own department's average is a correlated subquery. The average must be recomputed per employee using that employee's department, so the subquery references the outer row.

```
SELECT e1.* FROM Employee e1
WHERE salary > (
   SELECT AVG(salary) FROM Employee e2
   WHERE e2.department = e1.department
);
```

A correlated subquery re-executes once per row in the outer table, O(n) executions. On a large table, this is expensive. A non-correlated subquery executes once, and its result is reused, making it significantly more efficient.

Part 2: Long Answer

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

A LEFT JOIN keeps all customers regardless of whether a match exists in Order; filtering WHERE o.order_id IS NULL then isolates exactly those with no match. An INNER JOIN would silently drop unmatched customers, which is the opposite of what we want.

(b) Average order value per restaurant, only those with more than 5 orders

```
SELECT r.restaurant_id, r.name, AVG(ov.order_value) AS avg_order_value
FROM Restaurant r
JOIN (
   SELECT o.order_id, o.restaurant_id,
        SUM(mi.price * oi.quantity) AS order_value
   FROM "Order" o
   JOIN OrderItem oi ON o.order_id = oi.order_id
   JOIN MenuItem mi  ON oi.item_id  = mi.item_id
   GROUP BY o.order_id, o.restaurant_id
) ov ON r.restaurant_id = ov.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(ov.order_id) > 5;
```

The inner subquery first computes a per-order total (SUM), which is then averaged in the outer GROUP BY; computing AVG directly without this intermediate step would average individual line-item values rather than whole-order totals, giving the wrong result. HAVING is used instead of WHERE because the row count filter applies to groups, not individual rows.

(c) Restaurants whose average order value exceeds the platform-wide average

```
WITH order_values AS (
   SELECT o.order_id, o.restaurant_id,
        SUM(mi.price * oi.quantity) AS order_value
   FROM "Order" o
   JOIN OrderItem oi ON o.order_id = oi.order_id
   JOIN MenuItem mi  ON oi.item_id  = mi.item_id
   GROUP BY o.order_id, o.restaurant_id
),
restaurant_avgs AS (
   SELECT restaurant_id, AVG(order_value) AS avg_order_value
   FROM order_values
   GROUP BY restaurant_id
   HAVING COUNT(*) > 5
)
SELECT ra.restaurant_id, r.name, ra.avg_order_value
FROM restaurant_avgs ra
JOIN Restaurant r ON ra.restaurant_id = r.restaurant_id
WHERE ra.avg_order_value > (SELECT AVG(order_value) FROM order_values);
```

A non-correlated subquery is appropriate here: the platform-wide average is a single scalar computed once and compared against every restaurant; it does not depend on the current row being evaluated. A correlated subquery would recompute that scalar once per restaurant, which is wasteful with no benefit.

(d) Most ordered menu item (by total quantity) per restaurant

```
SELECT restaurant_id, item_id, item_name, total_quantity
FROM (
   SELECT mi.restaurant_id,
        mi.item_id,
        mi.item_name,
        SUM(oi.quantity) AS total_quantity,
        RANK() OVER (
           PARTITION BY mi.restaurant_id
           ORDER BY SUM(oi.quantity) DESC
        ) AS rnk
   FROM OrderItem oi
   JOIN MenuItem mi ON oi.item_id = mi.item_id
   GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) ranked
WHERE rnk = 1;
```

A window function with PARTITION BY restaurant_id cleanly assigns a rank within each restaurant's item group without collapsing rows, letting the outer query simply filter on rnk = 1.

Limitation of GROUP BY + MAX for top-N per group: A naive approach of selecting MAX(total_quantity) per restaurant can identify the maximum value, but cannot simultaneously retrieve the other columns of the winning row (like item_id or item_name) without a correlated subquery or self-join, and if two items are tied at the maximum, MAX silently picks neither or requires extra handling to surface both ties correctly.

Part 3: Normalisation

Q1. Functional Dependencies

student_id determines student_name
course_id determines course_name, instructor_id
instructor_id determines instructor_name, instructor_office
(student_id, course_id) determines grade, enrollment_date

The composite key is (student_id, course_id).

Q2. Is it in 1NF?
Yes, assuming each column holds a single atomic value and there are no repeating groups, 1NF is satisfied. The table has a defined primary key (student_id, course_id).

Q3. Partial Dependency violating 2NF
student_id alone determines student_name, and course_id alone determines course_name; instructor_id, each depends on only part of the composite key, not the whole key.

Anomaly example: If a student drops all their courses and their enrollment rows are deleted, student_name is lost entirely, even though the student's identity is unrelated to any enrollment. This is a deletion anomaly.

Q4. Transitive Dependency violating 3NF
course_id determines instructor_id, and instructor_id determines instructor_name, instructor_office. So instructor_name and instructor_office depend on instructor_id, not directly on the primary key; that's a transitive dependency.
Anomaly: If an instructor changes offices, every course row for that instructor must be updated. Missing even one causes inconsistency

Q5. 3NF Decomposition
1)Student(student_id, student_name)
PK: student_id; separated out because student_id partially determined student_name.

2)Instructor(instructor_id, instructor_name, instructor_office)
PK: instructor_id; separated out because instructor_id transitively determined instructor_name, instructor_office.

3)Course(course_id, course_name, instructor_id)
PK: course_id, FK: instructor_id; separated out because course_id partially determined course_name, instructor_id.

4)Enrollment(student_id, course_id, grade, enrollment_date)
PK: (student_id, course_id), FKs: student_id, course_id; retains only attributes that genuinely depend on the full composite key.

Part IV: Design Justification

1)
In Part II(a), finding customers who never placed an order, I chose a LEFT JOIN with a NULL check over a NOT IN subquery. The key correctness concern was NULL handling: if the Order table contains even one NULL in customer_id, a NOT IN subquery returns no rows at all, because x NOT IN (... NULL ...) evaluates to UNKNOWN for every row. The LEFT JOIN approach is immune to this and is also more readable for expressing "find unmatched rows."

2)
Splitting Enrollment into four tables means that a simple report like "show student name, course name, and grade" now requires three or four joins, whereas the original single table answered it instantly. Every read-heavy query pays a join cost that did not exist before.

Real systems sometimes intentionally keep denormalised tables, such as in analytics or reporting databases, because reads vastly outnumber writes and join overhead matters more than update anomalies. The redundancy is accepted as a deliberate tradeoff for faster, simpler queries.